uCLINUX PORTING HOWTO

Monday 26th of July 2004, 04:30:56 pm. Posted in: **Embedded**. 8804 words.

Assembly level startup code:

uclinux/linux-2.4.x/arch/armnommu/kernel/head-armv.S

head-armv.S contains the 32-bit startup code for the ARM architecture. A basic environment is to be setup before control is passed to the C function start_kernel in uclinux/linux-2.4.x/init/main.c. Note however, since a bootloader exists, the initialization of the processor and essential peripherals (DRAM, FLASH) is done during startup of the bootloader; none of this is performed by uClinux.

Once the bootloader copies the linux binary to DRAM, control is passed to the function stext in this file. The assembly code in this file is independent of the machine used, however to accomodate machine dependent initialization the register r1 containing the machine type (defined in uclinux/linux-2.4.x/arch/armnommu/tools/mach-types) is used. Either the bootloader, or code in this file sets the r1 register. r1 must be loaded with the same unique number defined in the mach-types file.

The kernel stack is setup at location (init_task_union+8192) and the BSS section cleared. Additionally, the variables __machine_arch_type and processor_id are loaded with the values MACH_TYPE_ACME and ACME_S3C44B0X_PROCESSOR_TYPE. MACH_TYPE_ACME ofcourse, is derived from the file mach-types, while ACME_S3C44B0X_PROCESSOR_TYPE is defined to a unique value equivalent to cpu_val in the __*_proc_info structure in uclinux/linux-2.4.x/arch/armnommu/mm/proc-arm6,7.S. In this case ACME_S3C44B0X_PROCESSOR_TYPE is 0x01040104.

```
/*
* Kernel startup entry point.
*
* The rules are:
* r0 - should be 0
* r1 - unique architecture number
```

```
* MMU - off
* I-cache - on or off
* D-cache - off
*
* See linux/arch/arm/tools/mach-types for the complete
* list of numbers for r1.
*/
.section ".text.init",#alloc,#execinstr
        .type      stext, #function
ENTRY(stext)
        mov        r12, r0

#if defined(CONFIG_ARCH_L7200)
/*
* FIXME - No bootloader, so manually set 'r1' with our architecture number.
*/
        mov        r1, #MACH_TYPE_L7200
...
...
...
#elif defined(CONFIG_ARCH_ACME)
        mov        r1, #MACH_TYPE_ACME
#endif

        /* make sure svc mode */
        mov        r0, #F_BIT | I_BIT | MODE_SVC
        /* and all irqs disabled */
        msr        cpsr_c, r0

#if defined(CONFIG_ARCH_ACME)
        adr        r5, LC0
        /* Setup stack */
        ldmia      r5, {r5, r6, r8, r9, sp}
```

```
        /* Clear BSS */
        mov         r4, #0
1:      cmp         r5, r8
        strcc       r4, [r5],#4
        bcc         1b

        /* Pretend we know what our processor code is
         (for arm_id) */

        ldr         r2, ACME_S3C44B0X_PROCESSOR_TYPE

        str         r2, [r6]
        mov         r2, #MACH_TYPE_ACME
        str         r2, [r9]

        mov         fp, #0
        b           start_kernel

LC0:    .long       __bss_start
        .long       processor_id
        .long       _end
        .long       __machine_arch_type
        .long       init_task_union+8192

ACME_S3C44B0X_PROCESSOR_TYPE:
        .long       0x01040104
#endif /* CONFIG_ARCH_ACME */
```

uclinux/linux-2.4.x/arch/armnommu/mm/proc-arm6,7.S

Assembly code to perform cache operations and TLB (translation lookaside buffer) functions are defined for the ARM7 core in this file. A processor info structure (__*_proc_info) is to be defined in this file for the new machine. This structure is used to setup the processor in the function setup_processor, accessed by uclinux/linux-2.4.x/init/main.c: start_kernel -> uclinux/linux-

2.4.x/arch/armnommu/kernel/setup.c: setup_arch -> uclinux/linux-2.4.x/arch/armnommu/kernel/setup.c: setup_processor. The proc_info_list structure is declared in uclinux/linux-2.4.x/include/asm-armnommu/procinfo.h:

```
struct proc_info_item {
        const char         *manufacturer;
        const char         *cpu_name;
};
struct proc_info_list {
        unsigned int         cpu_val;
        unsigned int         cpu_mask;
        unsigned long        __cpu_mmu_flags;
        /* used by head-armv.S */
        unsigned long        __cpu_flush;
        /* used by head-armv.S */
        const char           *arch_name;
        const char           *elf_name;
        unsigned int         elf_hwcap;
        struct proc_info_item *info;
#ifdef MULTI_CPU
        struct processor     *proc;
#else
        void                 *unused;
#endif
};
```

The structure __acme_proc_info is shown below. The logical AND of the variable processor_id (loaded in uclinux/linux-2.4.x/arch/armnommu/kernel/head-armv.S as described in the previous section) and the cpu_mask field should yield cpu_val in the proc_info_list structure, i.e: 0x01040104 & 0xfffff000 = 0x01040000. The object info of type proc_info_item is loaded with the manufacture and processor name.

```
cpu_acme_manu_name:
        .asciz       "Samsung"
```

```
cpu_acme_name:
        .asciz       "s3c44b0x"
        .type        cpu_acme_info, #object
cpu_acme_info:
        .long        cpu_acme_manu_name
        .long        cpu_acme_name
        .size        cpu_acme_info, . - cpu_acme_info


        .type        __acme_proc_info, #object
__acme_proc_info:
        .long        0x01040000        @ cpu_val
        .long        0xfffff000        @ cpu_mask
        .long        0x00000c1e        @ __cpu_mmu_flags
        b            __arm7_setup      @ __cpu_flush
        .long        cpu_arch_name     @ arch_name
        .long        cpu_elf_name      @ elf_name
        .long        HWCAP_SWP | HWCAP_26BIT      @ elf_hwcap
        .long        cpu_acme_info                @ info
        .long        arm7_processor_functions     @ info
        .size        __acme_proc_info, . - __acme_proc_info
```

uclinux/linux-2.4.x/arch/armnommu/kernel/entry-armv.S

IRQ related macros with processor specific knowledge are defined in this file. get_irqnr_and_base places the interrupt number in irqnr. In this case the value of the register I_ISPR is stored in irqnr, from which the actual IRQ number is extracted.

```
...
...
...
#elif defined(CONFIG_ARCH_ACME)
        .macro disable_fiq
        .endm
```

```
        .macro get_irqnr_and_base, irqnr, irqstat, base, tmp

        ldr         \base, =rI_ISPR

        ldr         \irqnr, [\base]

        teq         \irqnr, #0

        .endm


        .macro irq_prio_table

        .endm


#elif defined(CONFIG_ARCH_SWARM)

...

...

...
```